

Terraform Task 6

1. Terraform Modules

- A module is a collection of .tf files used together
- Every Terraform configuration is a module
- Root module → main working directory
- Child module → reusable module called using module {}

→ Module Structure

- main.tf → resources
- variables.tf → input variables
- outputs.tf → outputs
- versions.tf → Terraform & provider versions

→ Calling a Module

- Uses module block
- Requires source argument

→ Module Inputs & Outputs

- Inputs passed using variables
- Outputs accessed as module.<name>.<output>

→ Module Sources

- Local path
- Terraform Registry
- Git repository
- S3 / HTTP

→ Module Best Practices

- Keep modules small and focused
- Do not hardcode values
- Avoid provider configuration in child modules
- Use version constraints
- Make modules environment-agnostic

2. Terraform Workspaces

- A workspace allows multiple state files for the same configuration
- Default workspace name: default
- Same code, different state

→ Why Workspaces?

- Manage multiple environments
- Quick switching between environments
- Reduce code duplication

→ Workspace Commands

- terraform workspace list
- terraform workspace new <name>
- terraform workspace select <name>

→ Workspace Usage

- Workspace name accessed using terraform.workspace
- Commonly used for environment-specific values

→ State Behavior

- Each workspace has its own state file
- Local backend → separate state files
- Remote backend → separate state keys

→ Limitations of Workspaces

- Not recommended for production
- Weak environment isolation
- Risk of accidental changes
- Same backend and credentials

→ Modules vs Workspaces

- Modules → code reusability
- Workspaces → state separation
- Modules organize infrastructure
- Workspaces separate environments (limited use)

Watch Terraform-05 video. (TASK_1)

1. Execute the script shown in the video. (TASK_6)

- Create Terraform directory and then create under that directory create 3 more directory
- Aws_instance, development, production
- Under aws_instance we have

Terraform Modules:

→ Module is a collection of configuration files in a directory

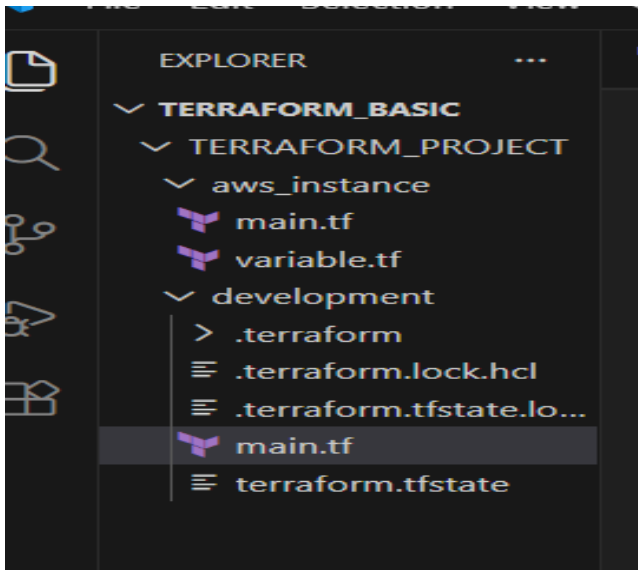
→ We can create Ec2 for different environments so that we can create a directory for

→aws_instance configuration

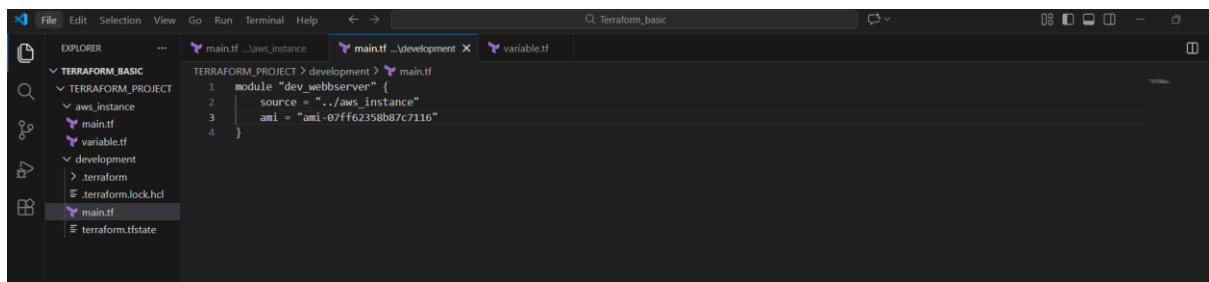
→main.tf

```
resource "aws_instance" "webserver" {  
  ami = var.ami  
  instance_type = "t2.micro"  
}
```

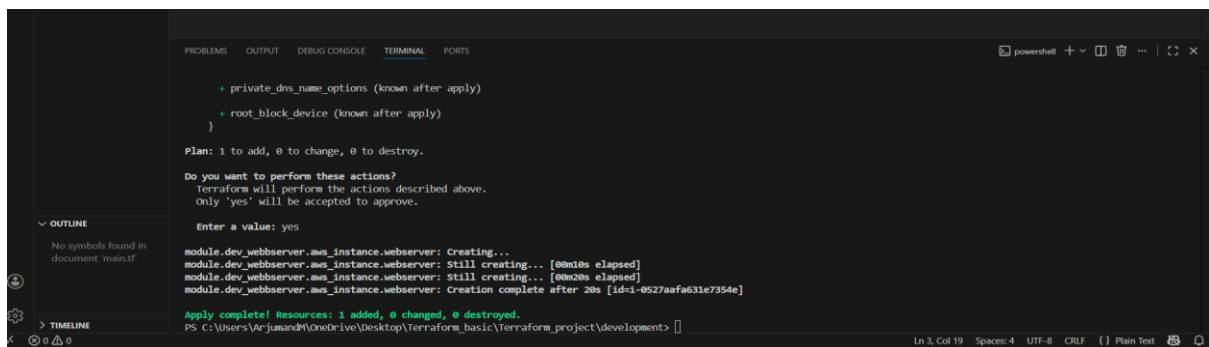
variable.tf
variable "ami" {



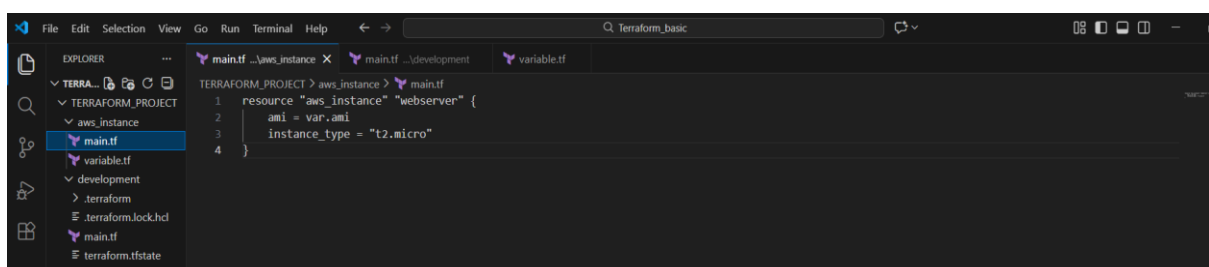
- After creating tables and files go to > development > main.tf and enter script



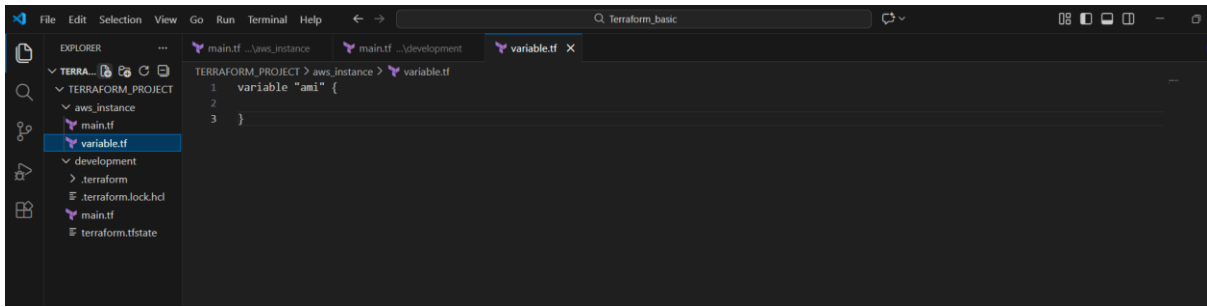
- Then enter terraform init and apply



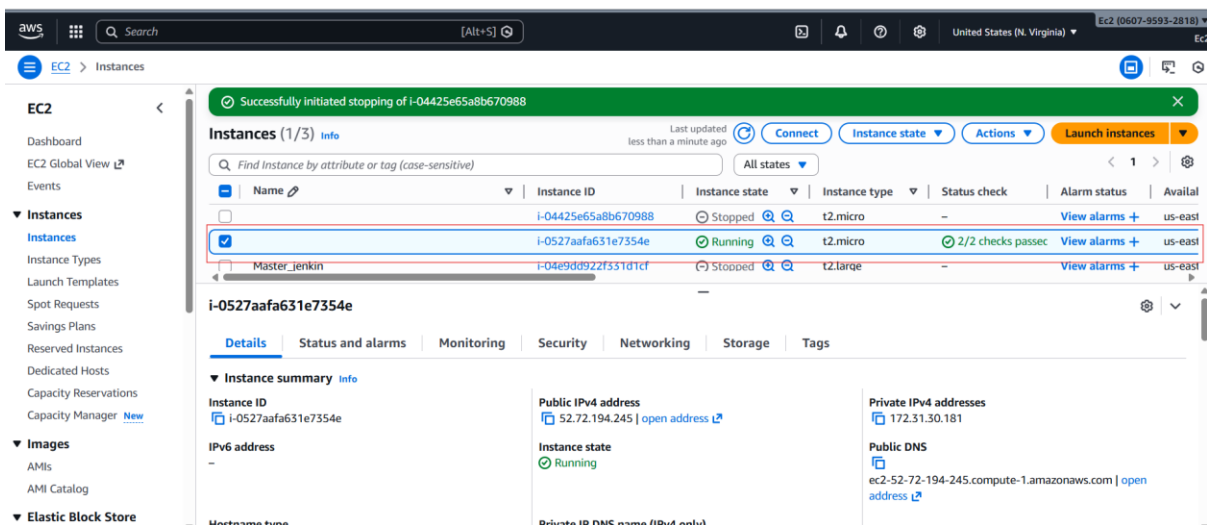
- This is what I have given in main.tf



- Variable.tf also created and not provided any value



- Instance was created in aws (Note: AWS configure required if it not done)



WORKSPACE:

Terraform Workspace

→ Workspace can be used if we have multiple project/environments using the same configurations

- terraform workspace new projectA ◇ to create a workspace names projectA
- terraform workspace list ◇ to list the workspaces available
- terraform workspace select projectA ◇ to switch to specific workspace

main.tf

```
resource "aws_instance" "webserver" {
  ami = var.ami
  instance_type = "t2.micro"
}
```

- Create workspace projectA
- Use command terraform workspace new projectA
- To check terraform worklist use command “terraform workspace list”

The screenshot shows the VS Code interface with a file explorer on the left showing a directory named 'TERRAFORM_PROJECT'. The main editor displays a Terraform configuration file 'main.tf' with the following content:

```
1 resource "aws_instance" "webserver" {
2   ami = var.ami
3   instance_type = "t2.micro"
4 }
```

The terminal at the bottom shows the following commands and output:

```
Directory: C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basic

Mode                LastWriteTime         Length Name
----                -
da---1            22-01-2026    20:23          TERRAFORM_PROJECT

PS C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basic> cd .\TERRAFORM_PROJECT\
PS C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT> terraform workspace new projectA
Created and switched to workspace "projectA"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT> terraform workspace list
 default
* projectA

PS C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT>
```

- Switch back to terraform default use command ‘terraform select default’

The screenshot shows the VS Code interface with the terminal at the bottom showing the following commands and output:

```
PS C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT> terraform workspace list
 default
* projectA

PS C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT> terraform workspace select default
Switched to workspace "default".
PS C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT>
```

- Write the exact script in main.tf

The screenshot shows the VS Code interface with the file explorer on the left showing a directory named 'TERRAFORM_BASIC'. The main editor displays a Terraform configuration file 'main.tf' with the following content:

```
1 resource "aws_instance" "webserver" {
2   ami = var.ami["projectA"]
3   instance_type = "t2.micro"
4 }
```

- Create Variable.tf and write exact script

```
variable.tf

variable "ami" {
  type = map(string)
  default = {
    "ProjectA" = "ami-07ff62358b87c7116"
```

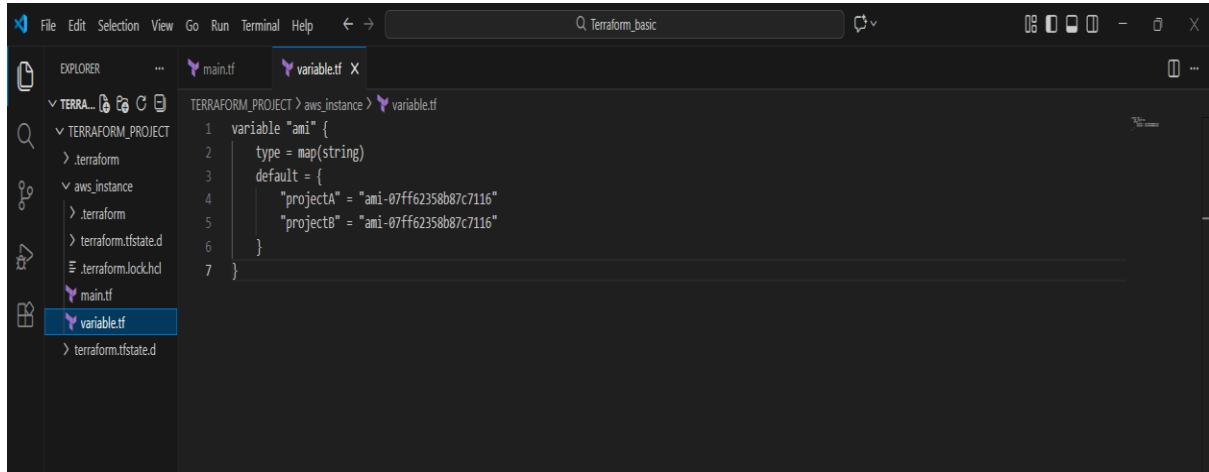
```
"ProjectB" = "ami-07ff62358b87c7116"
```

```
}
```

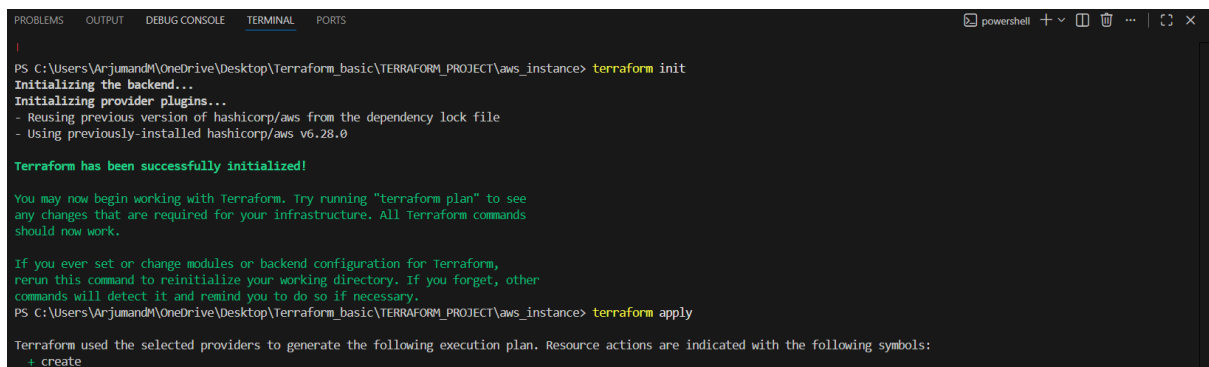
```
}
```

```
( terraform workspace new projectA < to create a workspace names projectA >
```

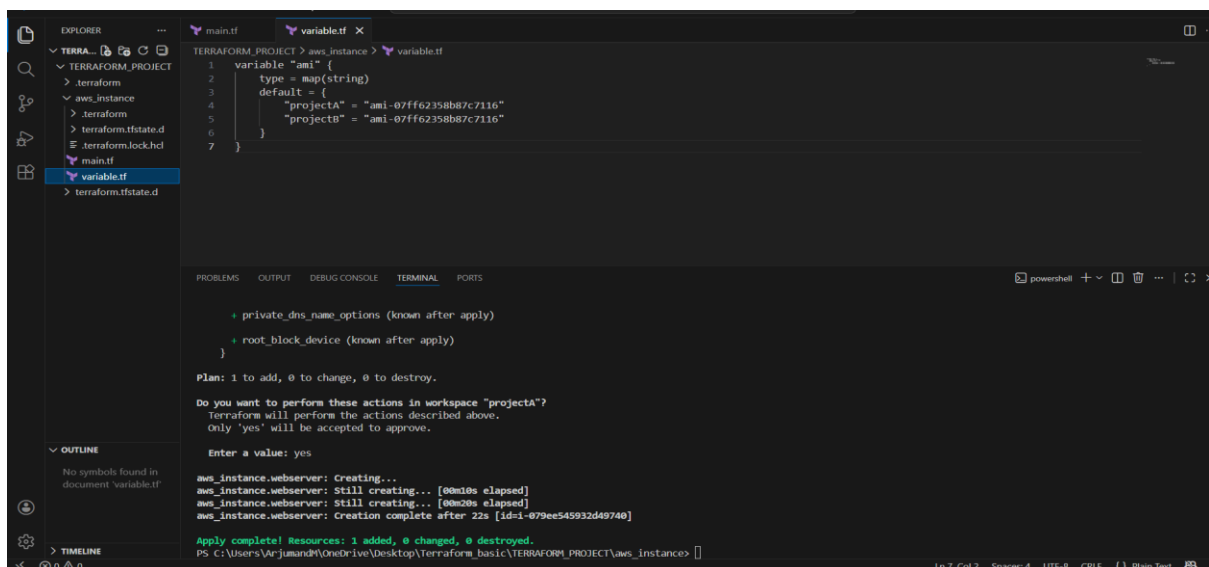
```
( terraform workspace list < to list the workspaces available >
```



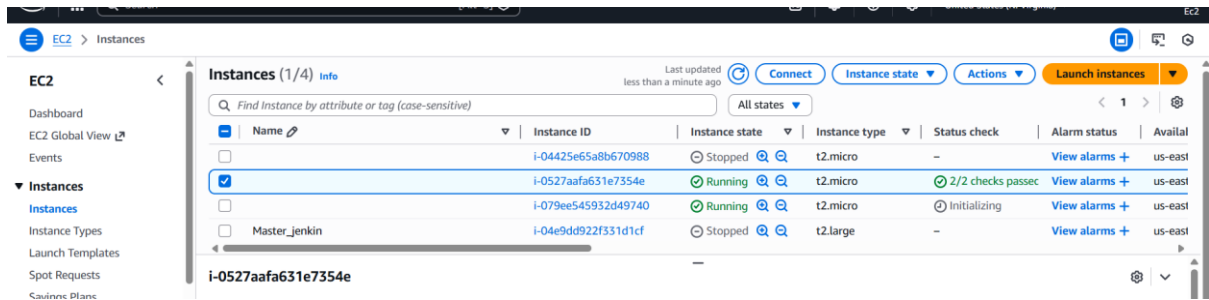
- Enter terraform init and terraform apply



- It will start creating instances



- Instance got created as we can check



3.Provision EC2, S3, and VPC using Terraform modules.

Main.tf

```
terraform {
```

```
  required_version = ">= 1.3.0"
```

```
  required_providers {
```

```
    aws = {
```

```
      source = "hashicorp/aws"
```

```
      version = "~> 5.63"
```

```
    }
```

```
  }
```

```
}
```

```
provider "aws" {
```

```
  region = "us-east-1"
```

```
}
```

```
# -----
```

```
# VPC MODULE
```

module "vpc" {

source = "terraform-aws-modules/vpc/aws"

version = "5.1.2"

name = "my-vpc"

cidr = "10.0.0.0/22"

azs = ["us-east-1a", "us-east-1b", "us-east-1c"]

private_subnets = ["10.0.0.0/26", "10.0.0.64/26", "10.0.0.128/26"]

public_subnets = ["10.0.1.0/26", "10.0.1.64/26", "10.0.1.128/26"]

enable_nat_gateway = true

single_nat_gateway = true

enable_vpn_gateway = false

tags = {

terraform = "true"

environment = "dev"

}

}

EC2 MODULE


```
# -----  
  
module "ec2" {  
    source = "terraform-aws-modules/ec2-instance/aws"  
    version = "4.3.0"  
  
    name = "my-ec2"  
  
    ami = "ami-07ff62358b87c7116"  
    instance_type = "t3.micro"  
  
    subnet_id = module.vpc.public_subnets[0]  
  
    associate_public_ip_address = true  
  
    tags = {  
        terraform = "true"  
        environment = "dev"  
    }  
}
```

```
# -----  
  
# S3 MODULE  
  
# -----  
  
module "s3" {  
    source = "terraform-aws-modules/s3-bucket/aws"
```

```
version = "4.1.2"
```

```
bucket = "shahbazz-s3"
```

```
tags = {
```

```
    terraform = "true"
```

```
    environment = "dev"
```

```
}
```

```
}
```

```
# -----
```

```
# OUTPUTS
```

```
# -----
```

```
output "vpc_id" {
```

```
    value = module.vpc.vpc_id
```

```
}
```

```
output "ec2_public_ip" {
```

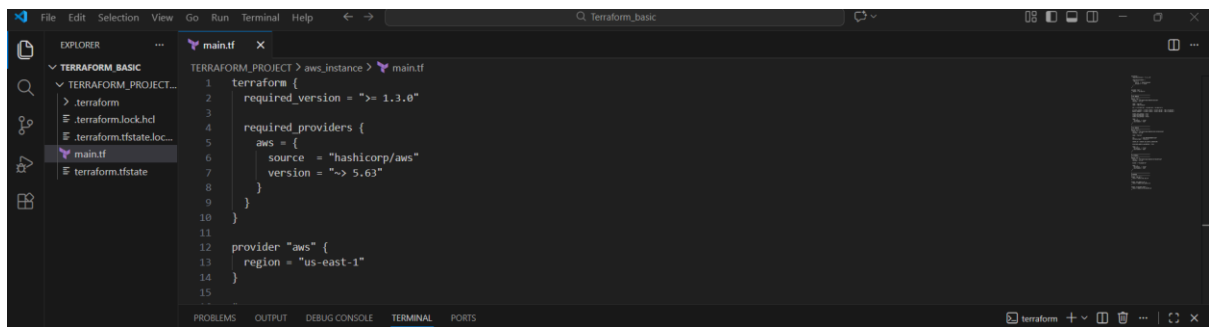
```
    value = module.ec2.public_ip
```

```
}
```

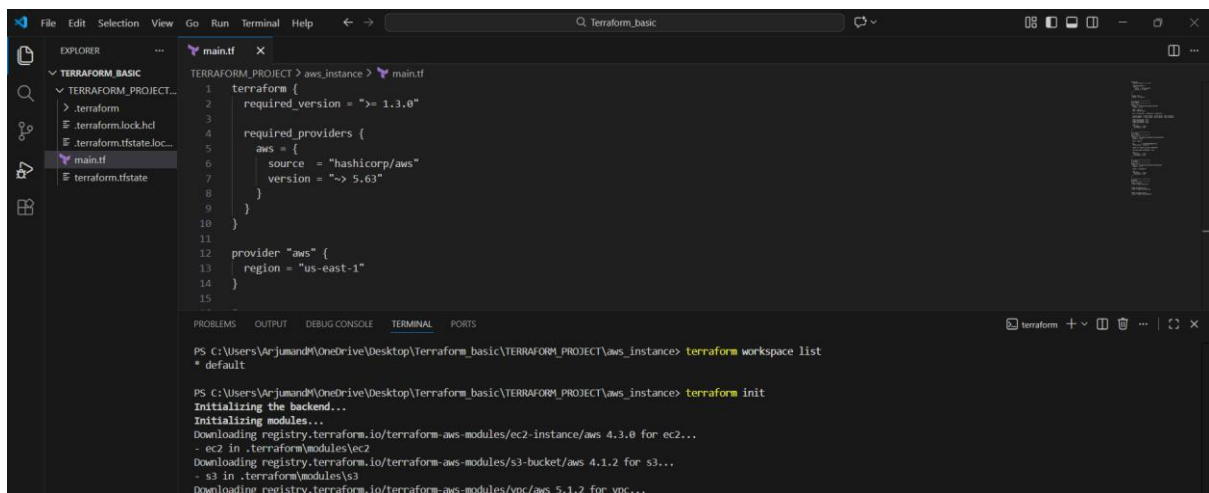
```
output "s3_bucket_name" {
```

```
    value = module.s3.s3_bucket_id
```

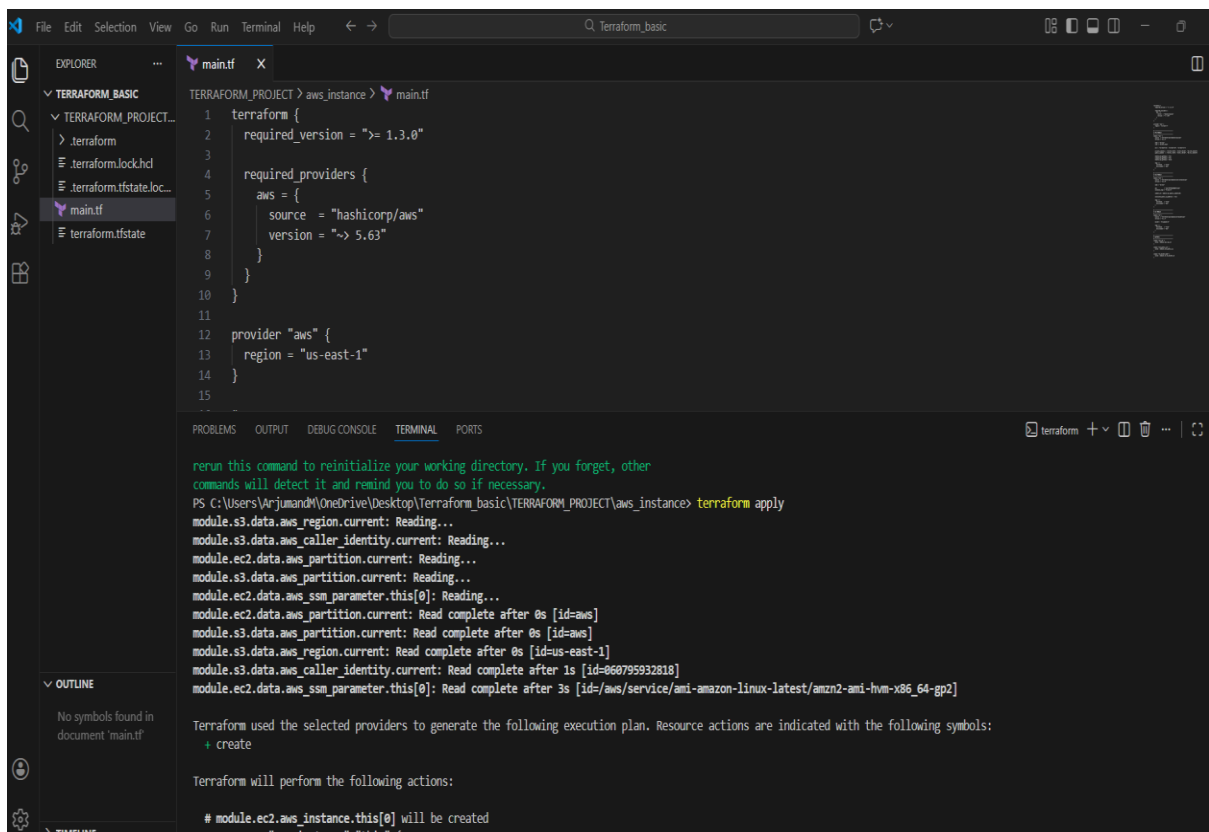
```
}
```



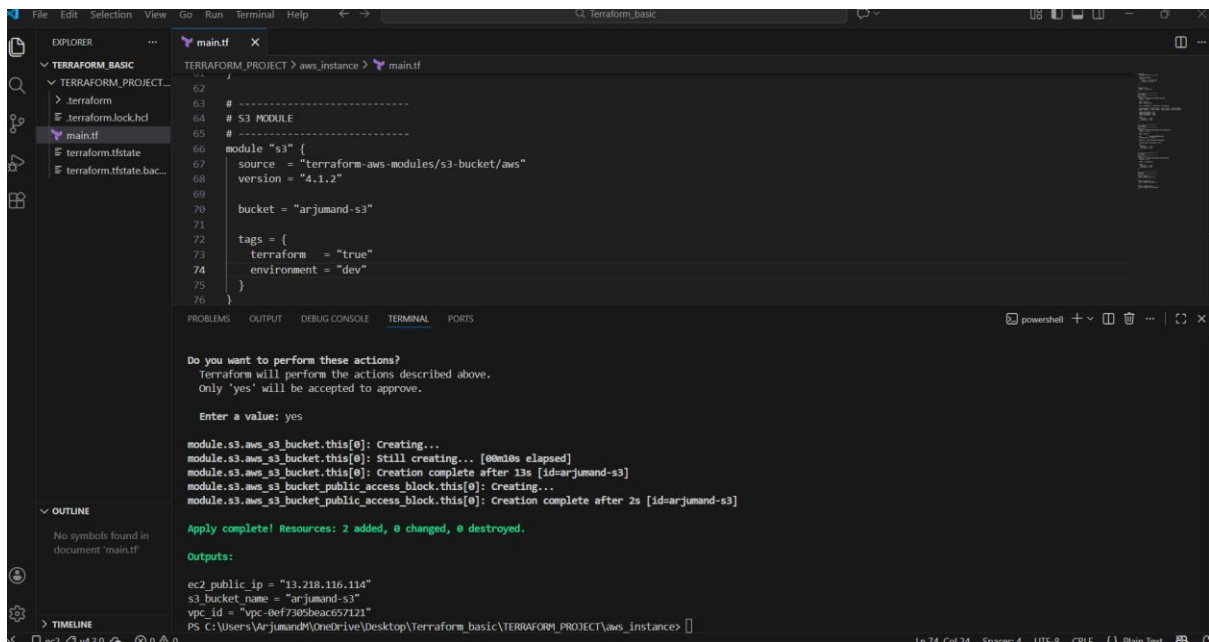
→ terraform init



→ Terraform apply



→ we can see Terraform output in terminal



```
main.tf
TERRAFORM_PROJECT > aws_instance > main.tf
62
63 # -----
64 # S3 MODULE
65 # -----
66 module "s3" {
67   source = "terraform-aws-modules/s3-bucket/aws"
68   version = "4.1.2"
69
70   bucket = "arjumand-s3"
71
72   tags = {
73     terraform = "true"
74     environment = "dev"
75   }
76 }

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

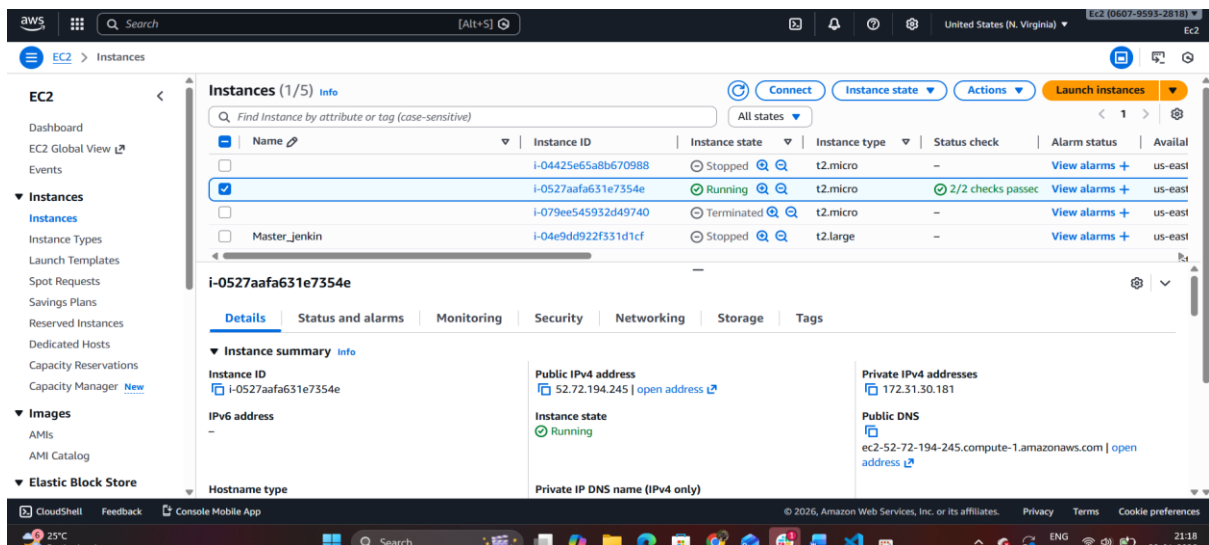
module.s3.aws_s3_bucket.this[0]: Creating...
module.s3.aws_s3_bucket.this[0]: Still creating... [00m10s elapsed]
module.s3.aws_s3_bucket.this[0]: Creation complete after 13s [id=arjumand-s3]
module.s3.aws_s3_bucket_public_access_block.this[0]: Creating...
module.s3.aws_s3_bucket_public_access_block.this[0]: Creation complete after 2s [id=arjumand-s3]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

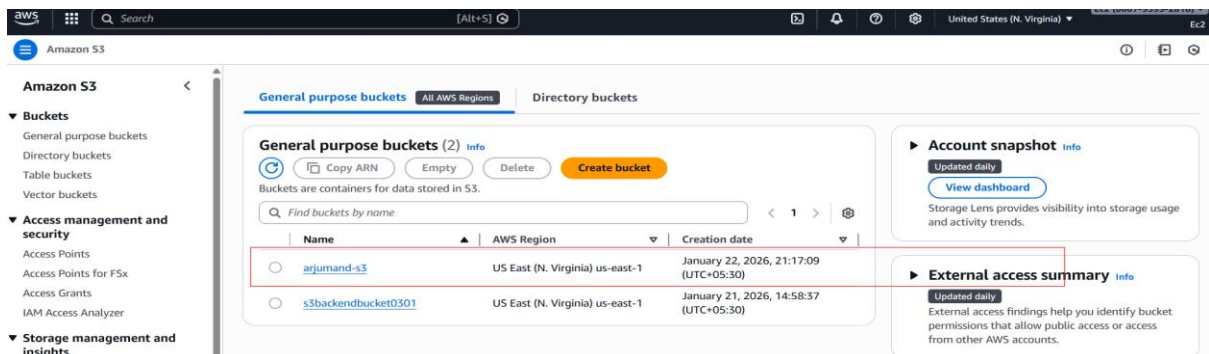
Outputs:

ec2_public_ip = "13.218.116.114"
s3_bucket_name = "arjumand-s3"
vpc_id = "vpc-0ef7305beac657121"
PS C:\Users\Arjumand\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance>
```

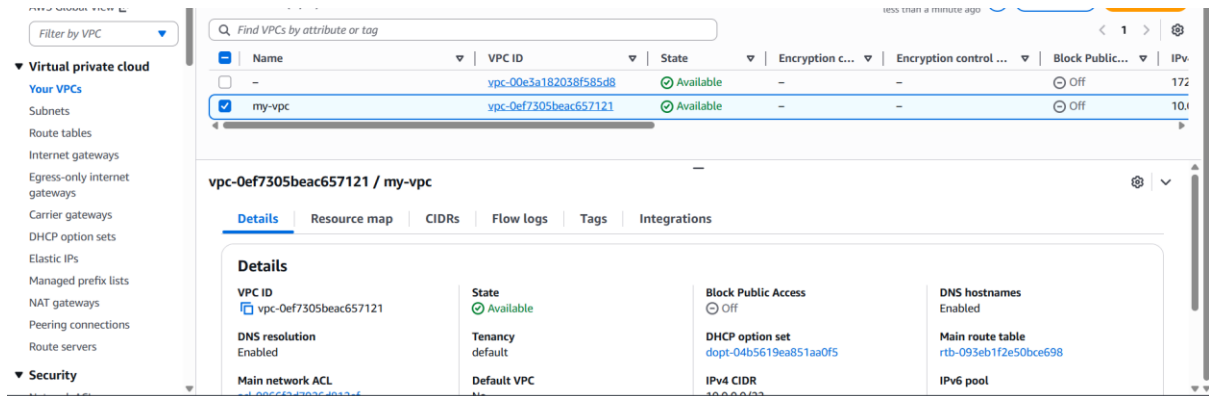
→ Now we can check in our AWS account for Ec2



→ And also we can check for S3 bucket with name < arjumand--s3 >



→ Check for VPC with name < my-vpc >



4. Provision EC2 for 3 different environments (Dev, Staging, and Prod) using Terraform workspace

Main.tf

```
terraform {
```

```
  required_version = ">= 1.3.0"
```

```
  required_providers {
```

```
    aws = {
```

```
      source = "hashicorp/aws"
```

```
      version = "~> 5.63"
```

```
    }
```

```
  }
```

```
}
```

```
provider "aws" {
```

```
  region = "us-east-1"
```

```
}
```

```
locals {  
  env = terraform.workspace
```

```
  
  instance_type_map = {  
    dev    = "t3.micro"  
    staging = "t3.small"  
    prod   = "t3.small"  
  }
```

```
  
  instance_name_map = {  
    dev    = "dev-ec2"  
    staging = "staging-ec2"  
    prod   = "prod-ec2"  
  }
```

```
}
```

```
# Default VPC
```

```
data "aws_vpc" "default" {  
  default = true  
}
```

```
# Default Subnet
```

```
data "aws_subnets" "default" {  
  filter {
```

```
    name = "vpc-id"

    values = [data.aws_vpc.default.id]

  }
}
```

Security Group

```
resource "aws_security_group" "sg" {

  name      = "${local.env}-sg"
  description = "Security group for ${local.env} "
  vpc_id    = data.aws_vpc.default.id

  tags = {

    environment = local.env

  }
}
```

EC2 Instance

```
resource "aws_instance" "app" {

  ami          = "ami-07ff62358b87c7116"
  instance_type = local.instance_type_map[local.env]

  subnet_id      = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.sg.id]

  tags = {
```

```

    Name      = local.instance_name_map[local.env]

    environment = local.env
}
}

```

```

output "environment" {

    value = local.env
}

```

```

output "instance_type" {

    value = aws_instance.app.instance_type
}

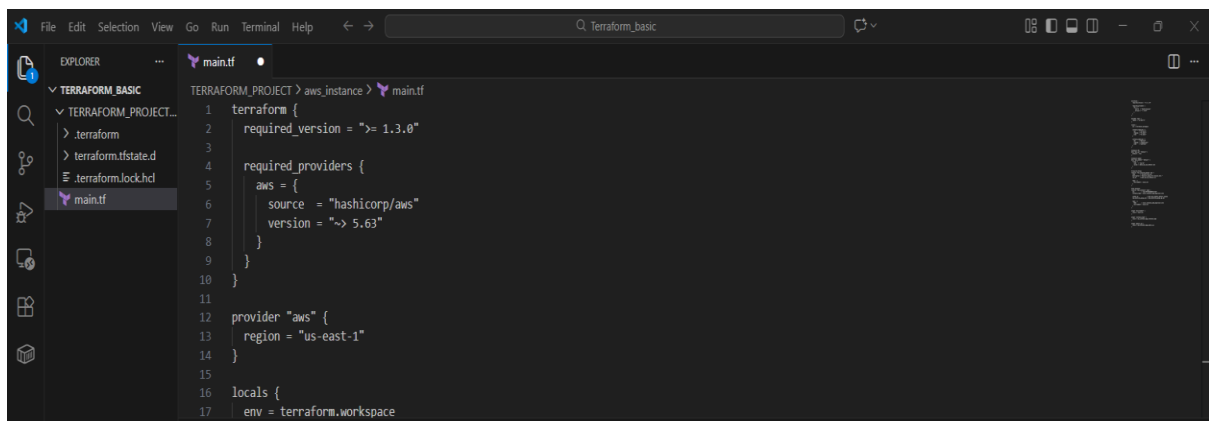
```

```

output "public_ip" {

    value = aws_instance.app.public_ip
}

```



➔ terraform init


```
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance>
* History restored

PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> cd .\TERRAFORM_PROJECT\aws_instance\
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.63"...

Error: Failed to query available provider packages

could not retrieve the list of available versions for provider hashicorp/aws: could not connect to registry.terraform.io: failed to request discovery document: Get
"https://registry.terraform.io/.well-known/terraform.json": context deadline exceeded

To see which modules are currently depending on hashicorp/aws and what versions are specified, run the following command:
    terraform providers

PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance> terraform init
Initializing the backend...
```

➔ Create 3 workspaces using command terraform workspace new dev/staging/prod

```
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance> terraform workspace new dev
Created and switched to workspace "dev"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance> terraform workspace new staging
Created and switched to workspace "staging"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance> terraform workspace new prod
Created and switched to workspace "prod"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance> terraform workspace list
default
* dev
* staging

PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance>
```

➔ Enter terraform apply after switching to dev workspace using command “tersform workspace select dev”

```
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance> terraform workspace select dev
Switched to workspace "dev".
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance> terraform apply
data.aws_vpc.default: Reading...
data.aws_vpc.default: Read complete after 2s [id=vpc-00e3a182038f585d8]
data.aws_subnets.default: Reading...
data.aws_subnets.default: Read complete after 0s [id=us-east-1]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.app will be created
+ resource "aws_instance" "app" {
  + ami              = "ami-07ff62358b87c7116"
  + arn              = (known after apply)
  + associate_public_ip_address = (known after apply)
```

➔ Enter terraform apply after switching to dev workspace using command “tersform workspace select staging”

```
environment = "dev"
instance_type = "t3.micro"
public_ip = "44.204.11.150"
PS C:\Users\Arjumand\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance> terraform workspace select staging
Switched to workspace "staging".
PS C:\Users\Arjumand\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance> terraform apply
data.aws_vpc.default: Reading...
data.aws_subnets.default: Read complete after 2s [id=vpc-00e3a182038f585d8]
data.aws_subnets.default: Read complete after 0s [id=us-east-1]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.app will be created
+ resource "aws_instance" "app" {
  + ami                    = "ami-07ff62358b87c7116"
  + arn                   = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone      = (known after apply)
  + cpu_core_count        = (known after apply)
  + cpu_threads_per_core   = (known after apply)
  + disable_api_stop       = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized          = (known after apply)
  + enable_primary_ipv6    = (known after apply)
  + get_password_data      = false
  + host_id                = (known after apply)
  + host_resource_group_arn = (known after apply)
}
```

➔ Enter terraform apply after switching to dev workspace using command “terrstrform workspace select prod”

```
Outputs:
environment = "staging"
instance_type = "t3.small"
public_ip = "34.239.115.178"
PS C:\Users\Arjumand\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance> terraform workspace select prod
Switched to workspace "prod".
PS C:\Users\Arjumand\OneDrive\Desktop\Terraform_basic\TERRAFORM_PROJECT\aws_instance> terraform apply
data.aws_vpc.default: Reading...
data.aws_vpc.default: Read complete after 2s [id=vpc-00e3a182038f585d8]
data.aws_subnets.default: Reading...
data.aws_subnets.default: Read complete after 0s [id=us-east-1]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.app will be created
+ resource "aws_instance" "app" {
  + ami                    = "ami-07ff62358b87c7116"
  + arn                   = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone      = (known after apply)
  + cpu_core_count        = (known after apply)
  + cpu_threads_per_core   = (known after apply)
  + disable_api_stop       = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized          = (known after apply)
  + enable_primary_ipv6    = (known after apply)
  + get_password_data      = false
}
```

➔ All three instances are created using different workspaces as we can check

☐	i-0527aafa631e7354e	 Running	t2.micro	 2/2 checks passed	View alarms +	us-east
☐	i-079ee545932d49740	 Terminated	t2.micro	-	View alarms +	us-east
☐	dev-ec2	 Running	t3.micro	 Initializing	View alarms +	us-east
☐	prod-ec2	 Running	t3.small	 Initializing	View alarms +	us-east
☐	staging-ec2	 Running	t3.small	 Initializing	View alarms +	us-east
☐	Master_jenkin	 Stopped	t2.large	-	View alarms +	us-east
☐	my-ec2	 Terminated	t3.micro	-	View alarms +	us-east

Select an instance